

Level erstellen in AR FLOW

In diesem Leitfaden erstellen wir ein neues Level in AR FLOW und gehen Schritt für Schritt durch, was dabei beachtet werden muss. Wir gehen von einem üblichen Labor-basierten Level mit einem Versuchsaufbau in einer Halle aus. Falls ein Level grundlegend anders aufgebaut ist, sind natürlich nicht alle Schritte gleich anwendbar (z.B. Potential Flow mit dem 3D-Diagramm oder Reynolds Dye Experiment mit dem 2D-Farbfaden).

Der Leitfaden arbeitet am Beispiel des TutorialLevel in **Assets/Scenes/TutorialLevel**. Für die Erstellung eines neuen Levels kann dieses TutorialLevel als Vorlage kopiert und angepasst werden.

Sprache

Im Allgemeinen haben wir uns darauf geeinigt, das gesamte Spiel in Englisch zu halten, somit auch die einzelnen Level.

1 Erste Schritte

Zunächst kann es nicht schaden, sich mit der Ordnerstruktur im Projekt vertraut zu machen. Da wir uns bemühen, das Projekt übersichtlich und aufgeräumt zu halten, sollte die Struktur selbsterklärend sein. So gehören innerhalb des **Assets**-Ordners z.B.:

- Bilder in **Images**
- 3D-Modelle in **Models**
- Code-Skripte in **Scripts**
- usw.

Insbesondere nach Abschluss des Leitfadens hilft es zudem, sich die bestehenden Level anzusehen und zu verstehen, wie sie aufgebaut sind. Es folgt eine kurze Übersicht der bereits vorhandenen Level:

- **Potential Flow**: 3D-SurfacePlot, um Potentialströmungen zu visualisieren.
- **3D Centrifugal Pump**: 3D-Modell einer Kreiselpumpe mit animierten Strömungslinien und verschiedenen Messgrößen.
- **Pump Curve**: Virtueller Versuchsstand, an dem die Pumpenkennlinie einer Kreiselpumpe bestimmt werden soll.
- **Pump Circuit**: Virtueller Versuchsstand, an dem die Lastkennlinie einer Waschmaschine bestimmt werden soll.

- **Reynolds' Dye Experiment:** Laminare und turbulente Strömung am Farbfadenversuch visualisieren.
- **Crossflow Valve:** Strömung und Druckverteilung in einem Crossflow-Ventil visualisieren.

Damit das zuvor erwähnte TutorialLevel im Hauptmenü erscheint, muss zunächst der entsprechende Haken gesetzt werden. In `Assets/Resources/Levels/TutorialLevel` befindet sich die Variable `showInMenu`, die standardmäßig auf `false` gesetzt (siehe Abb. 1). Schließlich wollen wir das TutorialLevel nicht dauerhaft im Hauptmenü anzeigen, sondern nur zu Testzwecken. Details zu den `LevelProperties` folgen in Abschnitt 3.1.

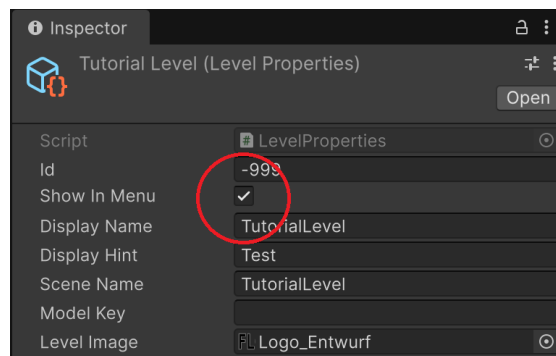


Abbildung 1: `LevelProperties` im Inspektor

2 Aufbau eines Levels

Grundsätzlich besteht ein Level aus einer Szene, und eine Szene ist immer ein Level (Ausnahme: `ARScene`, siehe unten). Eine Szene besteht üblicherweise aus zwei Teilen: den szeneneigenen Komponenten und dem einheitlichen `Pipemaster_Base`-Prefab (siehe Abb. 2).

Ausnahme: `ARScene`

Die `ARScene` ist kein eigenständig spielbares Level. Stattdessen übergibt man der `ARScene`, welches Modell in der Szene dargestellt werden soll. Somit können mehrere Level die gleiche `ARScene` verwenden, aber unterschiedliche 3D-Modelle anzeigen.

2.1 Einheitliches Prefab

Das `Pipemaster_Base`-Prefab enthält alle Komponenten, die in jedem Level gleich sind. Dazu gehören die drei Unterkomponenten `Pipemaster_Global`, `Pipemaster_Local` und

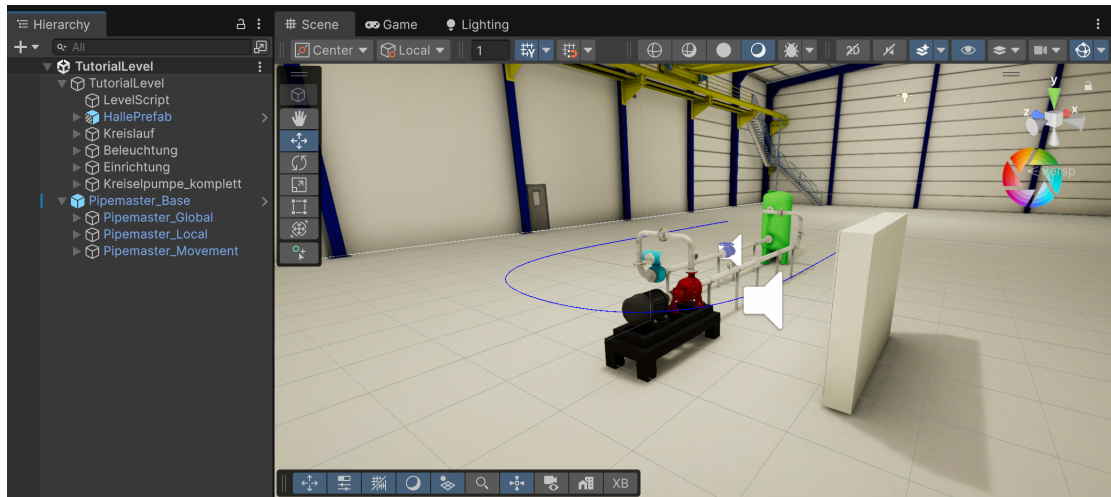


Abbildung 2: Aufbau eines Levels

Pipemaster.Movement. Alle drei Komponenten werden im Detail in den nachfolgenden Abschnitten beschrieben. Um ein Level zu erstellen, reicht es jedoch, dass dieses Prefab einfach in der Szene vorhanden ist.

Wenn manche Komponenten in der Szene nicht gewünscht sind, können sie einfach deaktiviert (nicht gelöscht) werden. Beispiel: In meinem Level brauche ich kein Hintergrundaudio, also deaktiviere ich das `GameObject BackgroundAudio` in `Pipemaster_Local`. Ansonsten reicht es in den allermeisten Fällen aus, das Prefab unverändert zu lassen und sich nur um die szeneneigenen Komponenten zu kümmern.

Niemals das Prefab überschreiben!

Das `Pipemaster_Base`-Prefab darf niemals überschrieben oder gelöscht werden, da es die Grundlage für alle Level bildet. Wenn man am Prefab Änderungen vornimmt (z.B. ein unbenutztes `GameObject` deaktiviert), dann wird diese Änderung vorerst nur in der aktuellen Szene übernommen. Dies ist soweit in Ordnung und gewollt. Wenn man jedoch das Prefab überschreibt (z.B. durch "Apply All", siehe Abb. 3), dann werden die Änderungen auf alle anderen Level übertragen. In der Regel ist dies nicht gewollt, im schlimmsten Fall kann dies zu unerwarteten Fehlern in anderen Leveln führen.

2.1.1 Pipemaster_Global

`Pipemaster_Global` beinhaltet alle Komponenten, die einmalig global im gesamten Spiel existieren. Während alle anderen Komponenten in jeder Szene neu instanziiert werden, sorgt `DontDestroyOnLoad()` dafür, dass auch beim Szenenwechsel dieselbe Instanz der

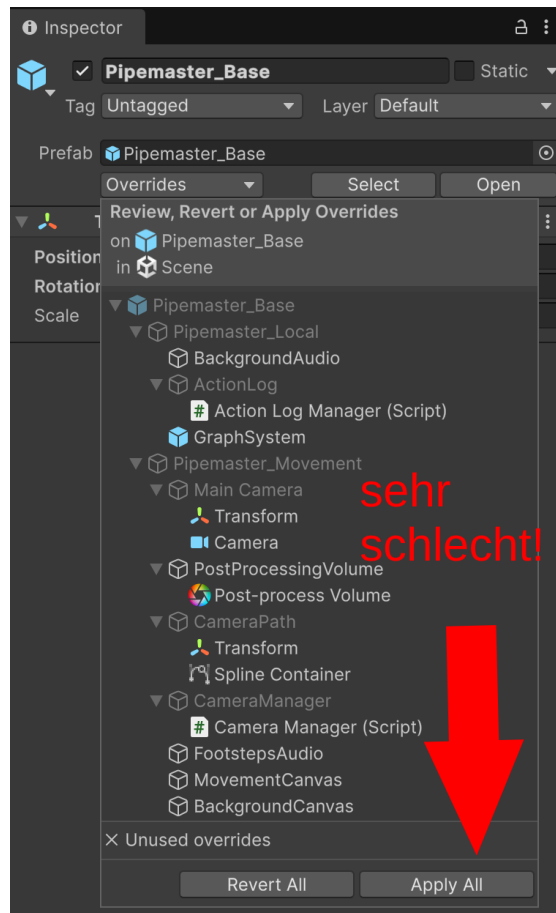


Abbildung 3: Niemals das Prefab überschreiben!

Komponenten in `Pipemaster_Global` erhalten bleibt. In der Praxis muss an diesem Teil nichts angepasst werden.

2.1.2 Pipemaster_Local

`Pipemaster_Local` beinhaltet alle Komponenten, die für jedes Level einzeln instanziiert werden. Grundsätzlich kann man den Teil auch unverändert lassen, bei Bedarf kann man jedoch einige Komponenten anpassen oder deaktivieren. Es folgt eine kurze Übersicht der Komponenten.

- **EventSystem:** Unity-eigene Komponente, bleibt unverändert.
- **ToolbarUI:** Zeigt die Toolbar sowie andere UI-Elemente an, die in jedem Level ähnlich sind. Bleibt unverändert.
- **QuestManager:** Verwaltet die Quests, bleibt unverändert.

- **InteractionMenuUI**: Zeigt das Interaktionsmenü für Objekte im Level an, bleibt unverändert.
- **BackgroundAudio**: Spielt die Hintergrundmusik ab. Bei Bedarf kann der Audio-Clip im **AudioSource** verändert oder das **GameObject** komplett deaktiviert werden.
- **ActionLog**: Protokolliert Aktionen im Spiel, bleibt unverändert.
- **GraphSystem**: Bietet die Möglichkeit, Graphen anzuzeigen. Wenn das Level keine Graphen benötigt, kann das komplette **GameObject** deaktiviert werden.

2.1.3 Pipemaster Movement

Pipemaster Movement beinhaltet alle Komponenten, die für die Bewegung innerhalb eines Levels zuständig sind. Genau wie bei **Pipemaster Local** werden alle Komponenten für jedes Level einzeln instanziiert. Es folgt eine kurze Übersicht der Komponenten.

- **Main Camera**: Unity-eigene Kamera. In der Theorie kann man Einstellungen an der Kamera vornehmen (z.B. FOV), in der Praxis wollen wir wahrscheinlich in jedem Level konsistente Kameraeinstellungen beibehalten.
- **PostProcessingVolume**: Post-Processing-Effekte für die Kamera. Wie bei der Kamera selbst: kann in der Theorie angepasst werden, in der Praxis wollen wir wahrscheinlich ein konsistentes Aussehen in allen Levels beibehalten.
- **CameraPath**: Definiert den Spline-Pfad, auf dem die Kamera entlangfährt. In den meisten Levels wird die Kamera entlang eines Pfads fahren, daher sollte diese Komponente in der Regel nicht deaktiviert werden. Allerdings muss der Pfad natürlich an das jeweilige Level angepasst werden.
- **CameraManager**: Verwaltet die Kamera-Logik und das Wechseln zwischen verschiedenen Kameramodi. In den jeweiligen Unterkomponenten können Anpassungen an den einzelnen Kameramodi vorgenommen werden. Im Folgenden werden nur die (Unter-)Komponenten aufgelistet, die relevante Anpassungen erfordern.
 - **CameraManager**: Initialer Kameramodus (**camType**), in laborbasierten Levels üblicherweise **SplinePath**.
 - **SplineCameraMode**: Initiale Position (**position**) der Kamera auf dem in **CameraPath** definierten Pfad.
 - **AerialCameraMode**: Rotation der Kamera in der Vogelperspektive, üblicherweise leicht nach unten (**cameraRotation**), und das imaginäre Rechteck in der Szene (**bounds**), in dem die Kamera bewegt werden kann. Schließlich wollen wir nicht, dass man in der Vogelperspektive aus der Halle entkommen kann 😊.

- **FootstepsAudio**: Spielt die Schrittgeräusche beim Bewegen der Kamera ab. Bei Bedarf kann der Audio-Clip im **AudioSource** verändert oder das **GameObject** komplett deaktiviert werden.
- **MovementCanvas**: Zeigt den Joystick an, bleibt unverändert.
- **BackgroundCanvas**: Zeigt das Hintergrundbild für die Detailansicht an, bleibt unverändert.

2.2 Szeneneigene Komponenten

Die szeneneigenen Komponenten sind alle Komponenten, die nicht Teil des einheitlichen Prefabs sind. Per Konvention fassen wir all diese Komponenten in einem **GameObject** zusammen, das den Namen der Szene trägt (siehe "TutorialLevel" in Abb. 2). Somit gibt es in jedem Level nur zwei Oberkomponenten, was der Übersichtlichkeit hilft.

Natürlich variiert die Anzahl und Art der szeneneigenen Komponenten von Level zu Level. Wenn man das **TutorialLevel** als Vorlage für ein neues Level mit einem Versuchsaufbau verwendet, kann man einige der Objekte natürlich behalten, wie z.B. die Halle und die Beleuchtung. Eine szeneneigene Komponente, die trotzdem in jedem Level vorkommt, ist das für jedes Level individuell erstellte **LevelScript**.

2.2.1 LevelScript

Jedes Level hat ein eigenes Skript, das die Logik für dieses Level enthält. Per Konvention befindet sich dieses Skript in **Scripts/LevelScripts**. Generell werden in dem Skript folgende Funktionen eingerichtet bzw. implementiert:

- Dem **QuestManager** wird die **QuestLine** für dieses Level übergeben, damit die Quests im Spiel angezeigt werden. Genaue Informationen zu Quests folgen in Abschnitt 2.3.1.
- Über den **ToolbarManager** wird eingestellt, welche Buttons in der UI angezeigt werden. UI-Elemente, die darüber gesteuert werden, sind:
 - Buttons in der Haupt-Toolbar unten rechts.
 - **LevelObjects**-Buttons oben links.
 - **FieldProperties** oben rechts, also die Anzeige der verschiedenen Messgrößen z.B. im "3D Centrifugal Pump"-Level.
 - Kamera-Button oben rechts, um in den laborbasierten Leveln zwischen verschiedenen Kameramodi zu wechseln.

Details zu den verschiedenen UI-Elementen folgen in Abschnitt 2.3.2.

- Es werden Referenzen zu allen relevanten `InteractionObjects` im Level gespeichert und die physikalischen Zusammenhänge zwischen den Objekten berechnet. Dies lässt sich gut am Beispiel der laborbasierten Level erkennen, z.B. in den Klassen `Pumpenkennlinie` und `ReihenParallelSchaltung` in `LevelScripts`. Details zu `InteractionObjects` folgen in Abschnitt 2.2.2.

Grundsätzlich ist das `LevelScript` des `TutorialLevel` sehr einfach aufgebaut, da das Level kaum Funktionalität hat. Nach Abschluss des Leitfadens ist es daher hilfreich, sich die `LevelScripts` der anderen Level anzusehen. Eine Übersicht aller Level findet sich in Abschnitt 1.

2.2.2 InteractionObjects

`InteractionObjects` sind alle Objekte in einem virtuellen Versuchsaufbau, mit denen der Spieler durch einen Klick auf das Objekt interagieren kann. Beim Klick auf das Objekt öffnet sich das entsprechende `InteractionMenu` (siehe Abb. 4). Jedes `InteractionObject` definiert für sich selbst, welche UI-Elemente im `InteractionMenu` angezeigt werden. Als Beispiel nehmen wir im Folgenden den Elektromotor, der außerhalb des `TutorialLevels` auch in den Levels "Pump Curve" und "Pump Circuit" vorkommt.



Abbildung 4: `InteractionMenu` des Elektromotors

Das `InteractionObject` für den Elektromotor ist in der Klasse `MotorObject` im Ordner `InteractionObjects` definiert. Grundsätzlich erben alle `InteractionObjects` von der Basisklasse `InteractionObject`. Durch die Basisklasse haben somit alle `InteractionObjects` zwei Attribute, die in der Szene für jedes Objekt im Inspektor gesetzt werden müssen (siehe Abb. 5):

- **name:** Anzeigename in der UI

- `questObject`: `QuestObject`, das mit diesem `InteractionObject` verknüpft ist. Details folgen in Abschnitt 2.3.1.

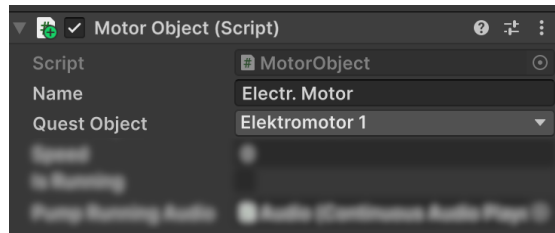


Abbildung 5: `MotorObject` im Inspektor

In den Unterklassen wird entsprechend das objektspezifische Verhalten definiert, z.B. welche UI-Elemente im `InteractionMenu` angezeigt werden. Dies ist am Beispiel des `MotorObject` an dem mehrfachen Aufruf von `AddMenuElement()` zu erkennen. Die Reihenfolge des Hinzufügens der Elemente bestimmt entsprechend die Reihenfolge der Anzeige. Alle möglichen UI-Elemente, die sich in einem `InteractionMenu` anzeigen lassen, befinden sich in `InteractionMenus/MenuElements`.

Konventionen für `InteractionMenus`

Per Konvention zeigen standardmäßig alle `InteractionObjects` als erstes Element im `InteractionMenu` einen Button an, der die Detailansicht für dieses Objekt öffnet. Daher lässt sich wahrscheinlich in jeder Unterklasse von `InteractionObject` als erstes der Aufruf `AddMenuElement(new Button("Detail View", OpenDetailView))` finden.

Natürlich merkt sich jedes `InteractionObject` auch dessen eigenen Zustand. Am Beispiel des Elektromotors ist dies die eingestellte Drehzahl und ob der Motor an oder aus ist. Diese Zustandsvariablen werden oft im `LevelScript` abgefragt, um die physikalischen Zusammenhänge zwischen den Objekten zu berechnen.

Abschließend registrieren sich `InteractionObjects` oft im `QuestManager`, um auf Quest-Interaktionen zu reagieren. So möchte man z.B. bei Erreichen einer bestimmten Quest bestimmte Menüpunkte blockieren. Im Falle des Elektromotors verbieten manche Quests dem Spieler, die Drehzahl zu verändern. Erst bei Erreichen einer anderen Quest wird der Drehzahl-Slider wieder freigeschaltet. Details zu Quests und Quest-Interaktionen folgen in Abschnitt 2.3.1.

2.2.3 Beleuchtung

Jede Halle braucht ihre Beleuchtung, so hat das `TutorialLevel` z.B. drei Lichtquellen unter `Beleuchtung`. Allgemein gibt es zwei Möglichkeiten, wie Beleuchtung in einem Level funktionieren kann:

- **Statisch**, d.h. die Beleuchtung wird vorab berechnet ("gebacken") und ist dann in der Szene fest.
- **Dynamisch**, d.h. die Beleuchtung wird in Echtzeit berechnet.

Während der Entwicklung eines Levels ist es sinnvoll, die Beleuchtung bei allen Lichtern auf dynamisch umzustellen (Mode -> Realtime, siehe Abb. 6). Somit kann ein Level sofort getestet werden, ohne dass vorher die Beleuchtung gebacken werden muss. Ansonsten müsste man nach jeder Änderung der Szenengeometrie die Beleuchtung neu backen, damit die Schatten und Schattierungen weiterhin korrekt sind, was auf meinem Laptop z.B. je nach Szene ca. 10 Minuten dauert.

Beleuchtung auf dynamisch umstellen!

Zum Entwickeln und Testen des Levels die Beleuchtung bei allen Lichtern auf dynamisch umstellen (Mode -> Realtime), und erst wenn das Level fertig ist, wieder auf statisch umstellen.

Wenn das Level fertig ist, kann die Beleuchtung wieder auf statisch umgestellt werden (Mode -> Baked), damit das Level auch auf Handys flüssig läuft. Um die Lightmaps korrekt zu generieren, sind jedoch weitere Schritte notwendig, die in Abschnitt 3.2 genau beschrieben werden.

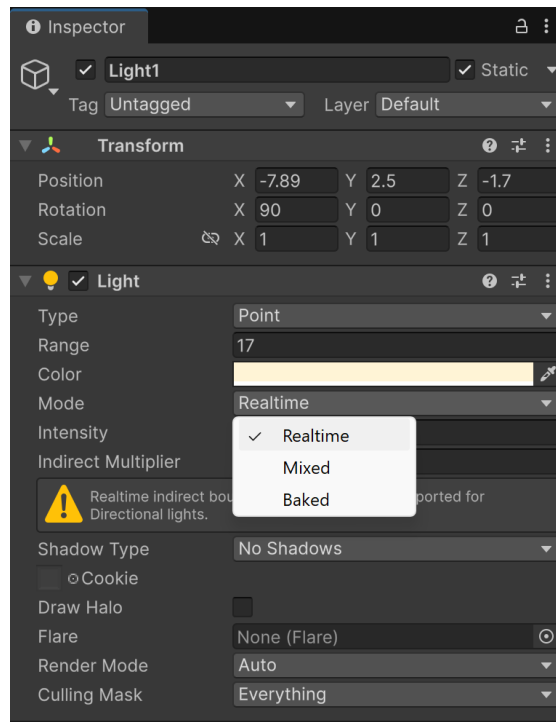


Abbildung 6: Beleuchtung auf dynamisch umstellen

2.3 Weitere Konzepte

In diesem Abschnitt werden weitere Konzepte eingeführt, die für die Levelentwicklung relevant sind, aber nicht direkt als `GameObject` in der Szene existieren.

2.3.1 Quests

Quests sind Aufgaben, die der Spieler im Level erfüllen soll. Sie leiten den Spieler durch das Level und helfen dabei, die Lernziele zu erreichen. Quests sind dabei geschachtelt definiert, d.h.:

- Eine `Quest` stellt eine einzelne Aufgabe dar, z.B. "Schalte die Pumpe an".
- Eine `QuestStage` ist eine Liste von Quests, die alle erfüllt werden müssen, um die `QuestStage` abzuschließen und die nächste `QuestStage` freizuschalten. Innerhalb einer `QuestStage` können die Quests in beliebiger Reihenfolge erfüllt werden.
- Eine `QuestLine` ist eine Liste von `QuestStages`, die nacheinander freigeschaltet werden. Sobald eine `QuestStage` abgeschlossen ist, wird die nächste `QuestStage` freigeschaltet. Eine `QuestLine` definiert somit die gesamte Quest-Struktur eines Levels, also $1 \text{ Level} = 1 \text{ QuestLine}$.

Eine Besonderheit des Quest-Systems ist, dass es zwischen dem Quest-System und dem Level an sich (`LevelScript`, `InteractionObjects`, etc.) hauptsächlich nur eine Art der Kommunikation gibt: `QuestInteractions`. Die Kommunikation über `QuestInteractions` funktioniert bidirektional:

- Objekte im Level schicken eine `QuestInteraction` an den `QuestManager`, um dem Quest-System mitzuteilen, dass eine bestimmte Aktion ausgeführt wurde.

Beispiel: Der Elektromotor schickt eine `QuestInteraction`, wenn dessen Drehzahl verändert wird. Der `QuestManager` prüft dann automatisch, ob diese Aktion relevant für eine Quest ist, und aktualisiert den Quest-Fortschritt entsprechend. Relevant heißt in diesem Fall, dass eine Quest exakt diese Interaktion als `QuestTarget` enthält.

- Quests und `QuestStages` schicken bei bestimmten Zustandswechseln (z.B. wenn eine Quest abgeschlossen wird) eine `QuestInteraction` als Broadcast raus. Alle interessierten Objekte im Level können sich im `QuestManager` für den Empfang registrieren, um auf diese `QuestInteractions` zu reagieren.

Beispiel: Wenn die Quest "Setze Drehzahl auf 1000 U/min" abgeschlossen wird, schickt die Quest eine `QuestInteraction` namens `BlockMotorSpeed` raus. Der Elektromotor empfängt diese `QuestInteraction` und blockiert den Drehzahl-Slider, bis eine andere Quest eine andere `QuestInteraction` rausschickt, bei dessen Empfang der Slider wieder freigeschaltet wird.

Eine `QuestInteraction` besteht immer aus drei Komponenten:

- Das `QuestObject`, mit dem die `QuestInteraction` verknüpft ist. In den meisten Fällen ist dies ein echtes Objekt in der Szene, z.B. der Elektromotor. Für jedes Objekt, das mit dem Quest-System interagiert, sollte innerhalb eines Levels auch ein einzigartiges `QuestObject` definiert sein. So gibt es z.B. `Elektromotor1` und `Elektromotor2`, für den Fall, dass in einem Level zwei Elektromotoren vorkommen.
- Der `QuestInteractionType` definiert die Art der Interaktion, also das *was*. Beispiele: `SetMotorSpeed` oder das zuvor erwähnte `BlockMotorSpeed`. Je nach Art der Interaktion können weitere Daten relevant sein, z.B. die eingestellte Drehzahl bei `SetMotorSpeed`. Der generische Typ des `QuestInteractionType` definiert somit, welchen Typ das Argument der `QuestInteraction` hat, z.B. `float` für die Drehzahl. Andere Interaktionstypen, wie z.B. `BlockMotorSpeed`, benötigen kein Argument, in diesem Fall ist der generische Typ `object` (`object` = nichts).
- Der Wert der Interaktion, also das Argument, z.B. die eingestellte Drehzahl bei `SetMotorSpeed`. Muss exakt dem Typ entsprechen, der im `QuestInteractionType` definiert ist, z.B. `float` bei `SetMotorSpeed`. Falls der Interaktionstyp kein Argument benötigt (= `object`), wird dieses Feld leer gelassen.

Sowohl `QuestObject` als auch `QuestInteractionType` werden ähnlich wie Enums verwendet, d.h. es gibt für beide Klassen eine vordefinierte Liste von möglichen Werten. Alle möglichen Werte für `QuestObject` befinden sich in `QuestObjects`, alle möglichen Werte für `QuestInteractionType` in `QuestInteractionTypes`. Dieser Ansatz wirkt vielleicht auf den ersten Blick komplex, doch er bietet Typgarantien und eliminiert die Gefahr von Tippfehlern bei Namen oder Interaktionsarten. Lohnt sich, vor allem bei einem immer weiter wachsendem Projekt!

Immer `QuestInteractions` verwenden!

In der Theorie bietet der `QuestManager` Callbacks wie `onQuestCompleted` an, die zwar auch genutzt werden können, um auf den Abschluss von Quests zu reagieren. Dann müsste das Objekt im Level jedoch z.B. den exakten Namen der Quest abfragen, um zu wissen, wann es reagieren soll. Dies ist sowohl fehleranfällig (z.B. Tippfehler im Questnamen) als auch unflexibel.

Daher, auch wenn ersteres auf den ersten Blick einfacher erscheinen mag: immer `QuestInteractions` (`QuestManager.onInteractionFromQuest`) verwenden, um auf Zustandswechsel von Quests zu reagieren!

Nachdem das Quest-System in der Theorie erklärt wurde, lassen sich nun die folgenden zwei Schritte durchführen, um Quests in einem neuen Level zu implementieren:

1. Erstellen einer `QuestLine`.
2. Registrierung und angemessene Reaktion auf Callbacks im `QuestManager`, v.a. um Quest-Interaktionen korrekt zu behandeln.

Eine `QuestLine` wird per Konvention immer als statische Variable in der statischen Klasse `QuestLines` definiert. Jede Instanz von `QuestLine` bildet exakt die Struktur der Quests ab, die in diesem Kapitel erklärt wurde. Ein kurzes Überfliegen der bereits definierten `QuestLines` sollte ausreichen, um die Syntax zu verstehen. Ansonsten sind folgende Kleinigkeiten relevant:

- **Name** und **Description** einer `QuestLine` ist der Text, der in der `LevelInfo-Box` angezeigt wird (`i`-Button unten rechts in der Toolbar, siehe Abb. 7).
- **Description** einer `QuestStage` kann leer gelassen werden, da sie aktuell nicht in der UI angezeigt wird. Eventuell finden wir dafür künftig einen passenden Platz in der UI, aber aktuell ist dieses Feld unbenutzt.

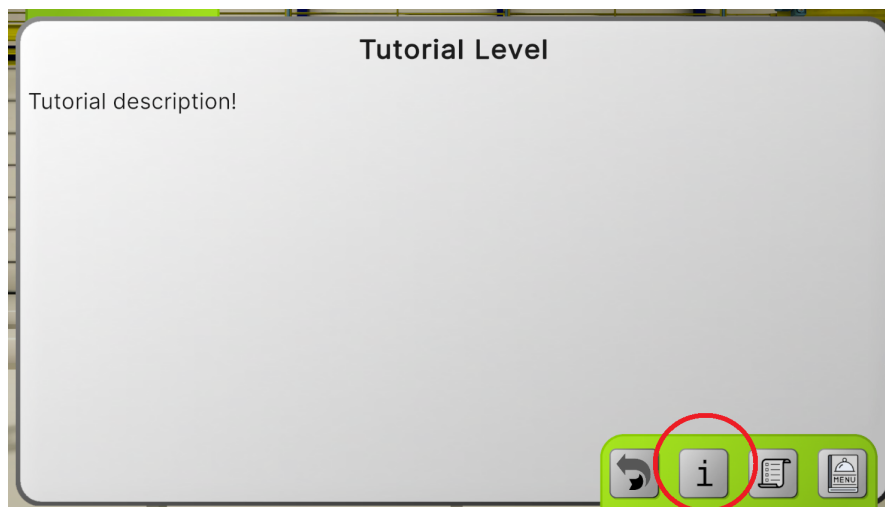


Abbildung 7: LevelInfo-Box

Um die fertig definierte `QuestLine` in der Level-Szene zu registrieren, muss sie im `LevelScript` der Szene dem `QuestManager` übergeben werden. Im Falle des `TutorialLevel` geschieht dies mit dem Aufruf `SetQuestLine(QuestLines.TutorialLevel)`. Eine solche Zeile findet man in jedem `LevelScript`, da jedes Level eine eigene `QuestLine` hat. Selbst wenn ein Level keine Quests nutzt, muss dennoch eine leere `QuestLine` definiert und übergeben werden, damit die `LevelInfo-Box` korrekt gefüllt wird.

Zum Schluss muss nur noch auf die Callbacks des `QuestManager` reagiert werden, falls es Quests gibt, die von sich aus eine `QuestInteraction` als Antwort zurückschicken. Als Erinnerung die bidirektionale Kommunikation:

$$\text{src} \xrightarrow{\text{QuestInteraction}} \text{QuestManager} \xrightarrow[\text{(optional)}]{\text{QuestInteraction}} \text{dest, where}$$

$$\text{src} = \text{dest} = \text{InteractionObject} \vee \text{LevelScript}$$

Wenn eine Quest eine `QuestInteraction` als Antwort zurückschickt, hängt es vom Kontext und von der Art der Interaktion ab, wo eine Reaktion darauf logisch sinnvoller ist. In der Praxis ist beides möglich, sowohl im `LevelScript` als auch in einem konkreten `InteractionObject`. In beiden Fällen ist `QuestManager.onInteractionFromQuest` der Callback, über den die `QuestInteraction` empfangen wird. Im `TutorialLevelLevelScript` ist ein Beispiel dafür zu finden.

Mit `IQuestInteraction.IsObjectAndType` wird überprüft, welche Interaktion konkret empfangen wurde. `IsObjectAndType` hat zwei Implementierungen: eine für Interaktionen ohne Parameter (z.B. `UseDetailView`), und eine für Interaktionen mit Parameter (z.B. `SetMotorSpeed`). Im ersten Fall wird nur `QuestObject` und `QuestInteractionType` überprüft, im zweiten Fall zusätzlich der Wert des Parameters der Interaktion als out-Parameter zurückgegeben.

2.3.2 UI-Elemente

Der `ToolbarManager` verwaltet eine Vielzahl an wiederverwendbaren UI-Elementen, die in den verschiedenen Levels genutzt werden können. Dazu gehört mehr als nur die Haupt-Toolbar unten rechts, wie in Abb. 8 zu sehen ist. Es folgen die wichtigsten UI-Elemente, die in den meisten Levels genutzt werden, sowie eine kurze Erklärung.

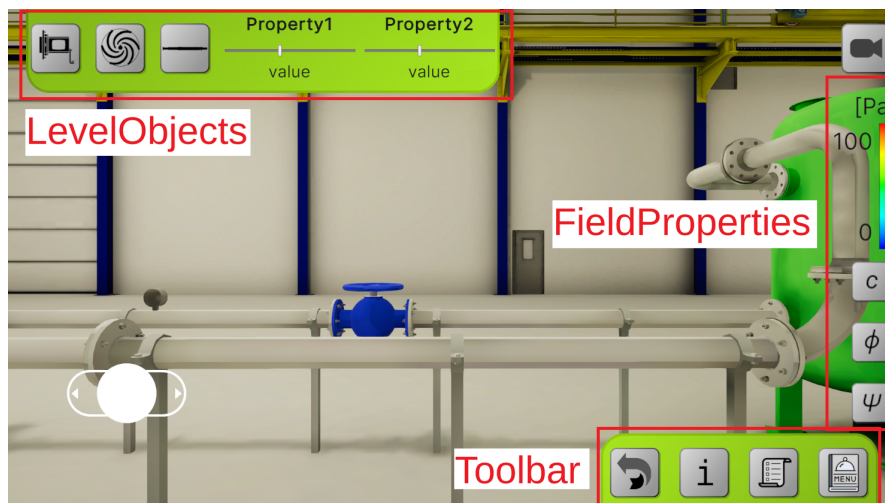


Abbildung 8: Wiederverwendbare UI-Elemente innerhalb eines Levels

- **Toolbar:** Haupt-Toolbar unten rechts mit Knöpfen, deren Funktionen über alle Level hinweg konsistent sein sollten.
- **LevelObjects:** Buttons oben links, die je nach Level komplett individuell belegt werden können.

- **FieldProperties**: Farbige Skala mit zusätzlichen Knöpfen, die als Legende für Messwerte genutzt werden kann.
- Kleinigkeiten wie eine **HintBox** für Tipps (`ShowHintWithText`) oder eine **FunctionBox**, um Funktionen im **Potential Flow**-Level anzuzeigen.

Für die Steuerung all dieser UI-Elemente stellt der **ToolBarManager** einheitliche Methoden zur Verfügung. In Verbindung mit den verschiedenen Enums und Hilfsklassen im selben Namespace (**ToolBarButton**, **ToolBarArea** etc.) können damit alle UI-Elemente in jedem Level auf einfache und konsistente Weise gesteuert werden. Ausnahme: die **Property1** und **Property2** Slider sind öffentliche Variablen im **ToolBarManager**, da sie bis jetzt nur in einem Level genutzt werden und sich somit keine lohnenswerten Abstraktionen anbieten.

Grundsätzlich sollten ein Blick auf den Code der Klasse und einige Beispiele in den **LevelScripts** ausreichen, um die Nutzung zu verstehen. Wenn man sich z.B. die zuvor genannten Enums anschaut, sieht man, dass es für fast alle Buttons und UI-Elemente einen Enum-Wert gibt. Dabei ist es dem gesunden Bauchgefühl überlassen, welche Buttons oder andere UI-Elemente man innerhalb eines Levels inwieweit anpassen sollte. Beispielsweise möchte man wohl nicht die Funktion des **MainMenu**-Buttons ändern oder die **ToolBar** vollständig ausblenden 😊.

2.3.3 Action Log

Der **Action Log** protokolliert alle relevanten Aktionen, die der Spieler im Spiel ausführt. Dafür zeigt er in der oberen linken Ecke des Bildschirms eine kurze Nachricht samt Emoji an, die nach 8 Sekunden automatisch wieder verschwindet (siehe Abb. 9). Das gibt dem Spieler hilfreiches Feedback zu seinen Aktionen, z.B. ob eine Interaktion erfolgreich war oder ob eine Quest abgeschlossen wurde. Wenn mehrere Nachrichten hintereinander verschickt werden, kann der Spieler durch alle Nachrichten wie in einer Liste scrollen.

Um die Nutzung zu vereinfachen, kennt der **ActionLogManager** nur eine öffentliche Methode: `Write(LogMessage message)`. **LogMessage** ist hierbei ein simples **struct**, das zwei Strings speichert: eins für das Emoji und eins für die Nachricht selbst. Per Konvention werden alle Nachrichten, die im gesamten Spiel genutzt werden, zentral in der Klasse **LogMessages** definiert. Wenn eine Nachricht dynamische Parameter erwartet, wird sie als statische Methode implementiert, ansonsten als **static readonly** Variable. Dies vereinfacht die Wiederverwendung von Nachrichten und stellt sicher, dass die Formulierungen und Emojis im gesamten Spiel konsistent sind.

Als der **Action Log** ursprünglich implementiert wurde, gab es einige Probleme mit der Anzeige von Emojis auf Android-Geräten. Daher musste mit **NotoColorEmoji** in **Assets/Fonts** eine eigene Schriftart eingebunden werden, damit die Emojis korrekt angezeigt werden. Seitdem müsste die Anzeige von Emojis auch auf Android problemlos funktionieren, aber dennoch:

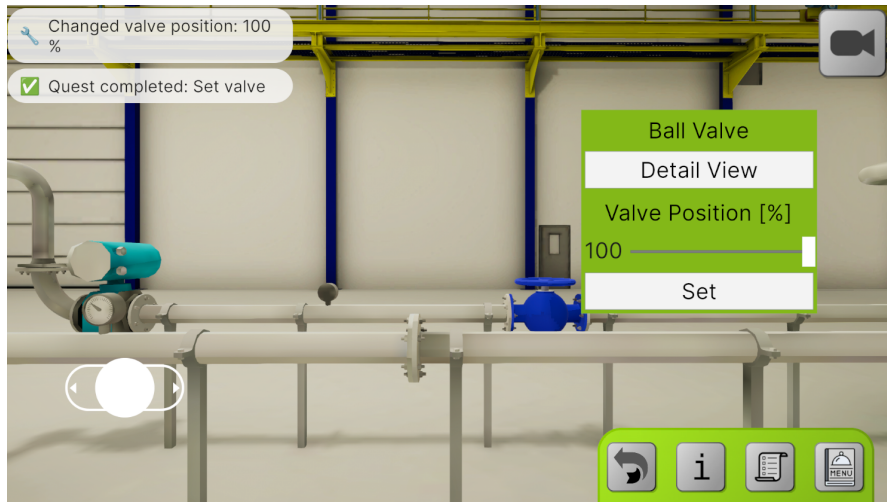


Abbildung 9: Beispielhafte Nachrichten im Action Log

Emojis im Action Log testen!

Wenn eine neue Nachricht in `LogMessages` definiert wird, die ein komplett neues Emoji verwendet, das noch in keiner vorhandenen Nachricht benutzt wird: unbedingt testen, ob das Emoji auch auf Android-Geräten korrekt angezeigt wird!

3 Einbetten des Levels

Die Szene, die zu dem neuen Level gehört, ist nun fertig! Es fehlen nur noch wenige Schritte, damit das Level auch vollständig in das Spiel integriert ist. Zunächst muss das Level im Hauptmenü registriert werden, damit es dort auch auswählbar ist (siehe Abschnitt 3.1). Anschließend sollte die statische Beleuchtung berechnet werden, damit das Level auch auf Mobilgeräten flüssig läuft (siehe Abschnitt 3.2).

3.1 Registrierung im Hauptmenü

Damit das neue Level im Hauptmenü auftaucht, muss ein neues `LevelProperties`-Objekt unter `Assets/Resources/Levels` angelegt werden. Alle `LevelProperties` in diesem Ordner werden durch den `MainMenuManager` ausgelesen, der daraus automatisch eine Level-Auswahl im Hauptmenü generiert. Das beispielhafte `LevelProperties`-Objekt des `TutorialLevels` wurde bereits in Abb. 1 gezeigt. Im Folgenden werden die einzelnen Felder kurz erläutert:

- **id:** Die ID des Levels, die zusätzlich für die Sortierung der Level im Hauptmenü verwendet wird. Das Level mit der niedrigsten ID wird in der Level-Liste als erstes angezeigt.
- **showInMenu:** Nur wenn dieser Haken gesetzt ist, wird das Level im Hauptmenü angezeigt.
- **displayName:** Der Name des Levels, der im Hauptmenü angezeigt wird.
- **displayHint:** Ein kurzer Hinweis, der in der Level-Liste über dem Namen des Levels angezeigt wird (z.B. "Level 1").
- **sceneName:** Der Name der Szene, die dem Level zugeordnet ist.
- **modelKey:** Key des AR-Modells, das in der AR-Szene geladen werden soll. Dieses Feld ist nur für die ARScene relevant und bleibt für alle anderen Level leer.
- **levelImage:** Das Vorschaubild für das Level, das im Hauptmenü angezeigt wird.

3.2 Statische Beleuchtung

Zuvor wurden in Abschnitt 2.2.3 kurz die zwei Arten von Beleuchtung in einer Szene erwähnt: statische und dynamische Beleuchtung. Aus Sicht des Entwicklers hat dynamische Beleuchtung den Vorteil, dass sie keinen zusätzlichen Implementierungsaufwand erfordert. Wir selbst müssen nichts einrichten, stattdessen berechnet Unity die Beleuchtung in Echtzeit, also jeden Frame neu.

Leider hat die Erfahrung aus den bereits existierenden laborbasierten Leveln gezeigt, dass dynamische Beleuchtung auf Mobilgeräten zu katastrophaler Performance führt. Unsere eigenen Handys am Fachgebiet Fluidtechnik sind dafür eine nützliche Referenz: das Samsung Galaxy A53 5G, ein Mittelklasse-Handy aus dem Jahr 2022. Es ist zwar weder das neueste noch das leistungsstärkste Handy, doch genau deswegen eine passende Basis, die wir mit unserem graphisch einfachen Spiel problemlos unterstützen sollten. Auf diesen Handys führte die Umstellung auf statische Beleuchtung zu einer 3-4x so hohen Framerate am Beispiel des Levels **Pump Circuit**: ca. 15 fps mit dynamischer Beleuchtung vs. 40-60 fps mit statischer Beleuchtung.

Dieses Beispiel zeigt, dass die Umstellung auf statische Beleuchtung für die Performance auf Mobilgeräten unerlässlich ist. Natürlich gilt dies nicht gleichermaßen für alle Level, stattdessen hängt es von der Komplexität der Szene in Bezug auf Modellgeometrie und Beleuchtung ab.

Nicht jedes Level benötigt statische Beleuchtung!

Laborbasierte Level mit vielen Modellen und Lichtern (Pump Curve, Pump Circuit) benötigen unbedingt statische Beleuchtung, damit sie flüssig laufen. Andere Level, die nur ein 3D-Modell beinhalten (Potential Flow) oder gar rein 2D sind (Reynolds' Dye Experiment, Crossflow Valve), laufen meist auch mit dynamischer Beleuchtung flüssig. Im Zweifel erst mit dynamischer Beleuchtung testen und nur wenn die Performance schlecht ist, auf statische Beleuchtung umstellen!

Die Umsetzung der statischen Beleuchtung gliedert sich in zwei Teile. In Abschnitt 3.2.1 bereiten wir die Szene für die Berechnung der statischen Beleuchtung vor. Im Anschluss wird in Abschnitt 3.2.3 erklärt, wie wir die Berechnung der statischen Beleuchtung anstoßen.

3.2.1 Vorbereitung der Szene

Damit die Szene vorbereitet ist, müssen zwei Schritte durchgeführt werden:

- Alle Lichter auf **Baked** setzen und Schatten einstellen
- Alle (statischen) Modelle auf **Static** setzen

Zunächst müssen alle Lichter in der Szene auf **Baked** gestellt werden (siehe Abb. 10). Falls es mehrere Lichter in der Szene gibt, kann es zusätzlich sinnvoll sein, dass nur eines der Lichter Schatten wirft. Dies wird durch die Einstellung **Shadow Type** erreicht, die im TutorialLevel nur für **Light2** auf **Soft Shadows** gestellt ist. Alle anderen Lichter sind auf **No Shadows** gestellt, damit die Schatten sauber aussehen. Falls ein neues Level sowieso in derselben Halle wie das TutorialLevel stattfindet, kann die Beleuchtung exakt so übernommen werden.

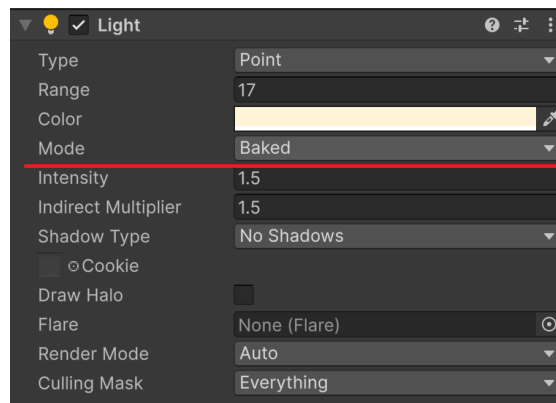


Abbildung 10: Beleuchtung auf statisch umstellen

Im zweiten Schritt müssen alle Modelle, die statisch beleuchtet werden sollen, auf **Static** gestellt werden. Dafür muss der entsprechende Haken oben rechts im Inspektor gesetzt sein (siehe Abb. 12). Falls Unity bei komplexen Objekten mit mehreren Unterobjekten danach fragt, sollen auch alle Unterobjekte auf **Static** gestellt werden. Zudem sollte sichergestellt werden, dass die Einstellungen im **MeshRenderer** bezüglich **Lighting** mit denen in Abb. 12 übereinstimmen, insbesondere die Einstellungen zu **Global Illumination**. Wenn zum Schluss die Lightmaps tatsächlich berechnet wurden, werden wie im Bild auch Informationen zur **Baked Lightmap** angezeigt, die zu diesem Modell gehört.

Ansonsten sollten in den Einstellungen des Modells selbst (nicht **MeshRenderer**) einige Anpassungen vorgenommen werden, um die Qualität der statischen Beleuchtung zu verbessern (siehe Abb. 11):

- Normals auf **Calculate** setzen
- Haken bei **Generate Lightmap UVs** setzen

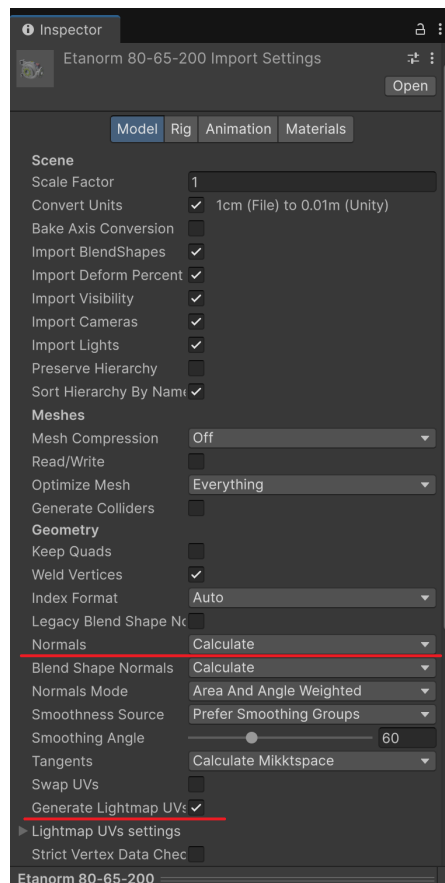


Abbildung 11: Inspektor eines 3D-Modells

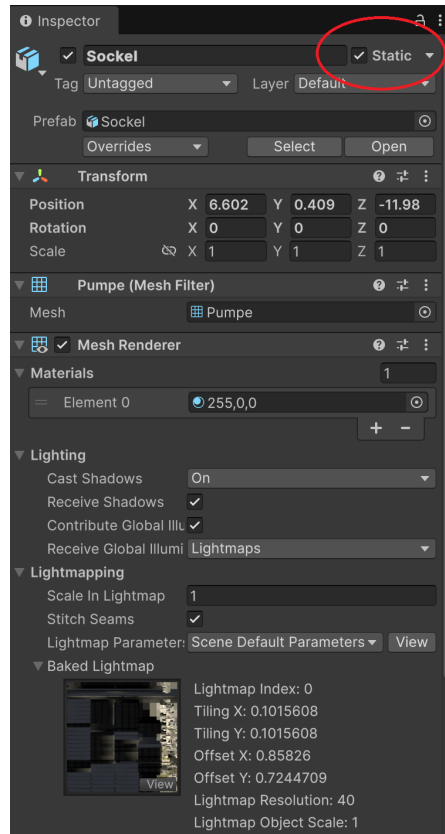


Abbildung 12: Modelle auf **Static** stellen

3.2.2 Alles statisch? Die Folgen von statischer Beleuchtung

Eine wichtige Implikation von statischer Beleuchtung ist, dass alle Modelle, die auf **Static** gestellt werden, ihren Beleuchtungszustand nicht mehr ändern können. Daraus folgt:

- Die Schatten, die von diesen Modellen geworfen werden, sind fest.
- Die Schattierung des Modells selbst, also dass z.B. die obere Seite des Modells heller als die untere Seite ist, ist ebenfalls fest.

Im besten Fall sollten statische Objekte somit nicht bewegt oder anderweitig animiert werden. In den jetzigen Leveln **Pump Curve** und **Pump Circuit** ist dies größtenteils der Fall. Der Versuchsaufbau selbst bewegt sich nicht, nur einige Einzelteile sind dezent animiert, wie z.B. das Ventil, wenn man die Ventilposition im Interaktionsmenü einstellt. Dafür muss jedoch der **Static**-Zustand des animierten Objekts, also des Ventilrads, angepasst werden (siehe Abb. 13). In diesem Fall muss der Haken bei **Batching Static**

entfernt werden. Ansonsten fasst Unity das gesamte Ventil intern zu einem Modell zusammen, sodass das Ventilrad-Unterojekt nicht mehr einzeln animiert werden kann. Damit ist zwar die Beleuchtung des Ventilrads weiterhin statisch, aber bei solchen dezenten Animationen ist das in der Regel kein Problem. Natürlich fällt bei genauer Betrachtung auf, dass sich die Schattierung des Ventilrads mitdreht. Solche kleinen Details sind jedoch eine akzeptable Verletzung der Faustregel, dass statische Objekte nicht bewegt werden sollten. Somit kann in Szenen mit überwiegend statischen Objekten, aber einigen wenigen animierten Teilen, weiterhin eine rein statische Beleuchtung verwendet werden.

Statische Objekte müssen sich (fast) statisch verhalten!

Alle Modelle, die auf **Static** gestellt werden, sollten sich nicht merkbar bewegen oder anderweitig animiert werden. Kleine Animationen fallen in der Regel nicht auf, aber bei größeren Bewegungen oder Animationen sollte eine **Mixed**-Beleuchtung erwogen werden.

Anders sieht es aus, wenn ein künftiges Level einen deutlich bewegbaren Gegenstand beinhaltet. In diesem Fall wird eine gemischte Lösung notwendig sein sein. Dabei werden alle Lichter auf **Mixed** gestellt, damit sie sowohl statische als auch dynamische Beleuchtung unterstützen. Für die Modelle, die deutlich bewegbar sein sollen, wird der **Static**-Haken entfernt, damit sie dynamisch beleuchtet werden. Die restlichen Schritte müssten identisch funktionieren, da aber kein Level zurzeit eine gemischte Beleuchtung verwendet, fehlen hierzu die Erfahrungswerte.

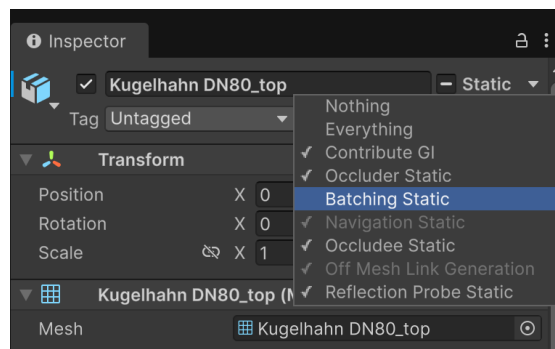


Abbildung 13: Detaillierte **Static**-Einstellungen

3.2.3 Backen der Beleuchtung

Nachdem die Szene vorbereitet ist, muss die statische Beleuchtung berechnet, also die Lightmaps gebacken werden. Dafür wählt im Menü **Window > Rendering > Lighting** die Registerkarte **Scene** aus. Dort findet sich der Button **Generate Lighting**, der die Berechnung der statischen Beleuchtung anstößt (siehe Abb. 14). Die restlichen Einstel-

lungen in diesem Fenster wurden bereits für eine Balance aus Qualität und Berechnungszeit optimiert, sodass sie nicht verändert werden müssen.

Je nach Komplexität der Szene kann die Berechnung einige Minuten dauern, bis zu 10 Minuten auf meinem Laptop. Sobald sie abgeschlossen ist, sollten die Informationen zu den **Baked Lightmaps** in den Inspektoren der statischen Modelle angezeigt werden, wie es bereits in Abb. 12 zu sehen war.

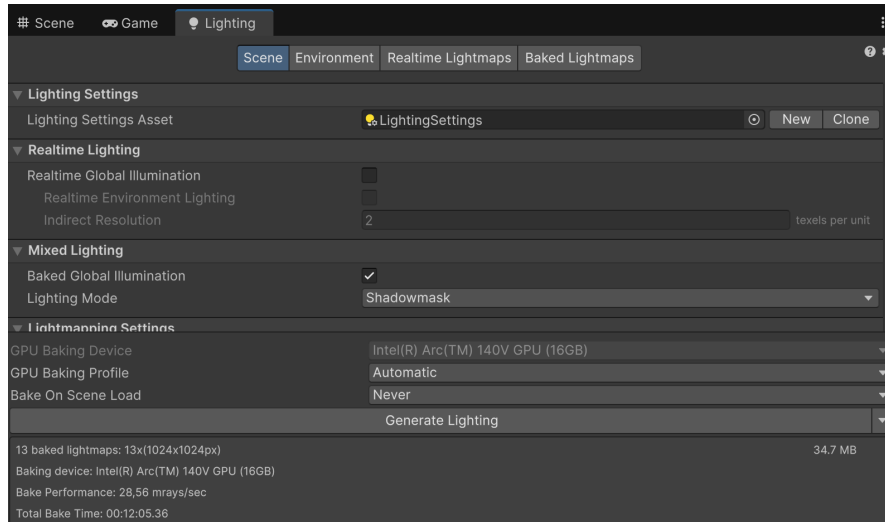


Abbildung 14: Lighting-Fenster zum Berechnen der statischen Beleuchtung

Szene verändert? Neu backen!

Nicht vergessen: Sobald die Szene verändert wird, z.B. durch das Hinzufügen eines neuen Modells oder das Verschieben eines bestehenden Modells, muss die statische Beleuchtung erneut berechnet werden, damit die Lightmaps aktualisiert werden. Bei großen Arbeiten am Level sonst die Beleuchtung temporär auf dynamisch stellen, bis die Szene fertig ist.

Zum Abschluss sollte bei allen Modellen überprüft werden, ob im **MeshRenderer** die **Baked Lightmap** korrekt angezeigt wird. Besonders bei Modellen mit großen Abmessungen kann es vorkommen, dass das Modell zu groß für eine Lightmap ist. Am Beispiel des Hallenflurs sieht man, dass Unity dafür im Inspektor eine Warnung anzeigt (siehe Abb. 15). In diesem Fall muss die Einstellung **Lightmap Scale** im **MeshRenderer** angepasst (z.B. halbiert) werden, bis die Warnung verschwindet und folglich das Modell in eine Lightmap passt. Wenn hingegen die Beleuchtung eines Modells zu fleckig oder verpixelt aussieht, kann die **Lightmap Scale** für einzelne Modelle auch erhöht werden.

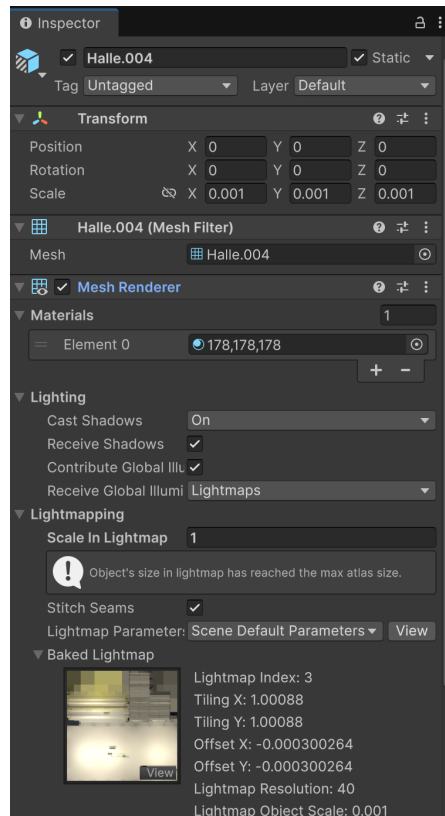


Abbildung 15: Modell zu groß für eine Lightmap